

NeuroAbs: A Neuro-Symbolic RTL Abstraction Framework for Property Checking Acceleration

Zhiyuan Yan¹, Xiaofeng Zhou², Ziyue Zheng¹, Ziyi Yang¹, Wenbin Che¹,
Wei Zhang², Yangdi Lyu¹, and Hongce Zhang^{1,2,*}

¹The Hong Kong University of Science and Technology (Guangzhou)

²The Hong Kong University of Science and Technology

*Corresponding author: hongcezh@hkust-gz.edu.cn

Abstract

Formal verification is a crucial technique for ensuring the functional correctness of hardware designs. In the context of property checking, a key challenge is how to efficiently prove a user-specified property in the face of increasingly complex RTL designs. To address this challenge, abstraction techniques are often employed to reduce system complexity and accelerate the verification process. However, prior RTL abstraction methods either require significant manual effort or rely on rule-based techniques that lack flexibility. This paper introduces NeuroAbs, a neuro-symbolic framework for RTL abstraction. NeuroAbs first uses LLM-assisted RTL analysis to identify signals suitable for abstraction. It then combines LLM-based abstraction with an AST-based symbolic RTL representation to better align the generated abstraction with the intended transformation. The soundness of each abstraction is checked using satisfiability modulo theories (SMT) solving. If the abstraction is too coarse for a successful proof, NeuroAbs applies counterexample-guided abstraction refinement (CEGAR) to iteratively refine the model. Experimental results show that NeuroAbs significantly improves the efficiency of hardware property checking across a range of verification tasks.

Keywords

Hardware formal verification, RTL abstraction, Neuro-symbolic reasoning

ACM Reference Format:

Zhiyuan Yan, Xiaofeng Zhou, Ziyue Zheng, Ziyi Yang, Wenbin Che, Wei Zhang, Yangdi Lyu, and Hongce Zhang. 2026. NeuroAbs: A Neuro-Symbolic RTL Abstraction Framework for Property Checking Acceleration. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '26)*, November 08–12, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3831252.3834042>

1 Introduction

Formal verification plays a crucial role in ensuring the correctness of digital circuit designs, particularly through property checking (also known as model checking). It systematically determines whether a given register-transfer-level (RTL) hardware model complies with a specified property. Upon property failure, a counterexample trace will be generated to illustrate how the property is violated by the given RTL design. Otherwise, the verification engine

will construct a formal proof establishing the property's validity. Over the years, several model checking algorithms have been proposed for hardware formal verification, including bounded model checking (BMC) [3], k-induction [21], and the property-directed reachability (PDR) algorithm [10].

As circuits grow in size and complexity, a major challenge in formal verification is how to efficiently determine the correctness of a given property. To address this, prior research has explored the use of abstraction techniques, which replace the concrete circuit model with a simplified representation that preserves only the essential behaviors relevant to verification. These techniques can be broadly classified into two categories: (1) manual abstraction, and (2) rule-based automatic abstraction. Manual abstraction largely relies on human insight into both the design and the target property. For instance, counter abstraction has been proposed to reduce the complexity of verifying systems that involve counters [18]. Similarly, Hsieh and Levitan [16] suggested abstracting digital circuit models using structural analysis and relational algebra. Bryant *et al.* [7] introduced the CLU (Counter Arithmetic with Lambda Expressions and Uninterpreted Functions) framework to model complex components such as infinite memories.

Rule-based abstraction, on the other hand, is an automated approach that systematically performs the abstraction process by applying predefined rules or algorithms. For example, Mishchenko *et al.* and Fan *et al.* utilized gate-level abstraction to accelerate the BMC and PDR algorithms, respectively [12, 19]. PDR-WLA abstracts the model by replacing signals generated by multipliers, adders, etc. with primary inputs [15], while IC3IA implements implicit predicate abstraction to perform the PDR algorithm at a higher level [8]. Furthermore, AVR [14], as the champion of the 2020's hardware model checking competition (HWMCC) [20], employed the syntax-guided abstraction, which implicitly builds the abstraction model by extracting syntactic terms directly from the word-level syntax of the given hardware description. In addition, AVR also performs datapath abstraction by replacing datapath operations with uninterpreted functions (UFs) to abstract complex arithmetic operations. Fan *et al.* proposed the datapath propagation technique to generate a more accurate abstract model in AVR for the datapath [11]. These techniques reduce the complexity of the digital model, enabling a more efficient verification process.

In general, however, how to apply proper abstraction is both design-specific and property-specific. For example, Fig. 1 shows a Verilog code snippet of the decoding and ALU logic in a microprocessor. In the formal verification of this design, the engineer may, for example, use assertions to check the correct RTL implementations for each type of instructions, which is the case in ISA-formal [17]. It



This work is licensed under a Creative Commons Attribution 4.0 International License. ICCAD '26, November 08–12, 2026, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2873-0/2026/11

<https://doi.org/10.1145/3831252.3834042>

```

1 module pipeline(input clk, input rst, input
2   [7:0] inst, ...);
3   wire [1:0] op, rs1, rs2, rd; reg [7:0] val [0:3];
4   wire id_wen; wire [7:0] alu_result;
5   assign op = inst[7:6]; assign rs1 = inst[5:4];
6   assign rs2 = inst[3:2]; assign rd = inst[1:0];
7   // Before Abstraction
8   assign id_wen = op == OP_ADD || op ==
9     OP_SUB || op == OP_AND;
10  // After Abstraction
11  assign id_wen = op == OP_ADD ? 1'b1 : 1'b0;
12  ...
13  // Before Abstraction
14  assign alu_result = op == OP_ADD ?
15    val[rs1] + val[rs2] : op == OP_SUB ?
16    val[rs1] - val[rs2] : op == OP_AND ?
17    val[rs1] & val[rs2] : 8'bxxxxxxx;
18  // After Abstraction
19  assign alu_result = op == OP_ADD ?
20    val[rs1] + val[rs2] : 8'bxxxxxxx;
21  assert property (...); // Whether the instruction
22    ADD can be correctly decoded and executed;

```

Figure 1: An example of abstraction based on the high-level information of the given property. In this example, `OP_ADD`, `OP_AND` and `OP_SUB` are three different parameters, where `OP_ADD` = 2'b01, `OP_SUB` = 2'b10, and `OP_AND` = 2'b11.

is not hard to see that, when verifying the property for one instruction type (for example, the ADD instruction), the RTL logic related to other instructions (e.g., bitwise AND and subtraction operations) can be safely abstracted. Specifically in this example, the assignment to the signal `id_wen` and `alu_result` can be simplified by replacing (part of) a concrete expression to a free value (which may also be referred to as unknown value or X-value), since the execution of these statements will not contribute to the outcome of the specified property. However, this kind of abstraction relies on a high-level understanding of the RTL design and the property specification. Traditional rule-based abstraction methods, such as structural abstraction and syntax-guided abstraction, usually are not “smart” enough to identify such opportunities per each verification setting, as they rely on predefined terms or syntactic rules rather than semantic reasoning about the property’s intent. Consequently, they fail to recognize abstraction opportunities that depend on functional understanding. More importantly, the abstraction opportunities in our setting depend jointly on the verification scenario, the target property, and the local RTL context, making them difficult to enumerate exhaustively with a fixed set of hand-written rules. On the other hand, manual abstraction construction remains tedious and labor-intensive.

To address the above challenge, we propose NeuroAbs, a neuro-symbolic framework for automated RTL abstraction. While LLMs are effective at generating candidate rewritings from design and property intent, a critical challenge is whether the generated abstraction remains aligned with the intended abstraction transformation. In particular, free-form rewriting lacks explicit structural constraints, and may therefore abstract logic beyond what is justified by the target property, modify logic outside the intended abstraction region, introduce syntactic and semantic errors, or violate the required over-approximation relation. NeuroAbs addresses these issues by combining LLM-based generation with an AST-based symbolic representation of RTL and formal checking. Specifically, NeuroAbs constructs the abstract model in three steps. First, it performs LLM-assisted analysis to identify the signals that are likely relevant to the target property. Second, it traverses the symbolic RTL representation to localize the corresponding statement-level AST nodes and uses another LLM to rewrite the selected RTL only

within these designated regions. Third, it uses SMT-based soundness checking and counterexample-guided abstraction refinement (CEGAR) [9] to ensure that the generated abstraction is a sound over-approximation and to refine it when it is too coarse. We evaluate NeuroAbs on several representative verification tasks, including those from RISC-V Formal [27], instruction-level abstraction refinement checking [13, 17], and the verification of an I2C peripheral. Experimental results show that NeuroAbs accelerates formal proof search and significantly improves bug-finding efficiency with BMC.

Overall, this paper makes the following contributions:

- We propose NeuroAbs, a neuro-symbolic framework for RTL abstraction. It uses one LLM to analyze design and property intent and identify abstraction candidates, and another LLM to generate the abstraction. To the best of our knowledge, it is the first work to apply LLMs to RTL abstraction.
- We introduce a neuro-symbolic abstraction flow in which the neural component proposes scenario-aware abstraction candidates from the design and property context, while the symbolic component constrains rewriting to AST-level targets, validates the generated abstractions through SMT checking, and iteratively refines the abstract model through a customized CEGAR procedure when the abstraction is too coarse.
- We evaluate NeuroAbs on a range of model checking problems and show that it improves verification efficiency across different tasks and algorithms.

2 Background

2.1 Hardware Model Checking and Abstraction

An RTL model can be represented as a finite state transition system: $M := \langle V, Init(V), Tr(V, V') \rangle$. Here, V represents state variables, and V' indicates next-state variables. $Init(V)$ specifies the initial states, and $Tr(V, V')$ defines the transition relation. The input variables in RTL are treated as free state variables which are not bounded by the transition relation, and therefore, we omit them in the formulation.

Given a safety property $P(V)$, the hardware model checking algorithm determines whether $P(V)$ holds for all states reachable from $Init(V)$. If $P(V)$ is valid for all reachable states, the property is safe, and a formal proof can be constructed. Otherwise, the system is regarded as unsafe, and a counterexample trace is produced.

To improve scalability, abstraction is commonly used to construct a model that focuses on the essential behaviors of the RTL while simplifying certain details. The abstract transition system is formally defined as $\hat{M} := \langle \hat{V}, \hat{Init}(V), \hat{Tr}(V, V') \rangle$, where $\hat{M}$ is the overapproximation of the original system M . The abstraction guarantees that $Init \models \hat{Init}$ and $Tr \models \hat{Tr}$, ensuring that the state space of $\hat{M}$ is strictly a superset of the state space of M . If the safety property $P(V)$ is proved as safe under the abstract model $\hat{M}$, it also implies that the M is safe. However, if $P(V)$ is unsafe under the $\hat{M}$, the generated counterexample may be spurious due to the too coarse abstraction. Therefore, the abstract model must be refined using the CEGAR method to eliminate this counterexample. As there could be more than one spurious counterexample, the process of abstraction and refinement may form a loop, commonly referred to as the CEGAR loop.

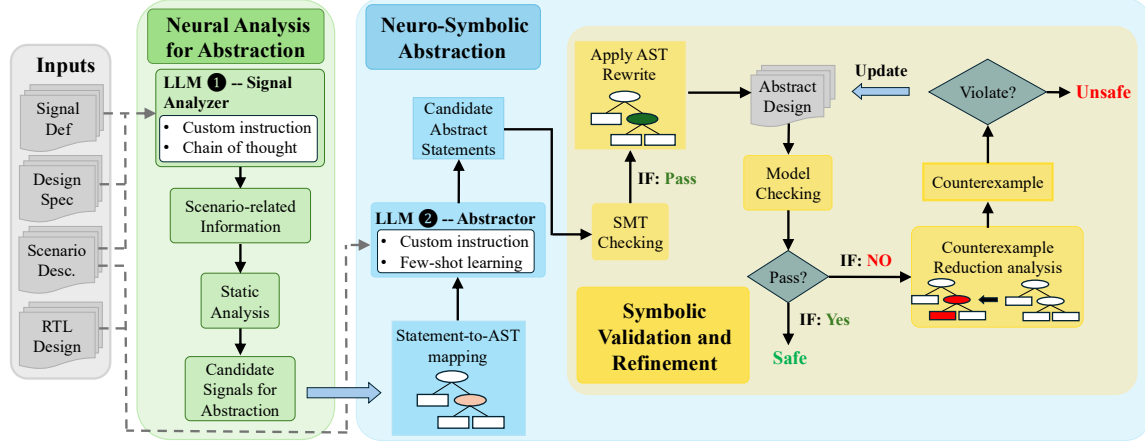


Figure 2: An overview of NeuroAbs.

2.2 Scenario-based Verification Setup

Modern hardware often supports multiple configurations or operating modes, each exhibiting distinct functional behaviors. These variations are typically verified separately, with each case requiring a customized setup. We refer to each such focused verification effort as a verification scenario, which defines the operational context, relevant assumptions, and the properties to be formally checked (i.e., property specification). For example, in processor verification, one scenario may target the correct execution of one instruction class (e.g., ALU instructions with immediates), while another may check consecution of program counters. Similarly, for a Serial Peripheral Interface (SPI) controller, verification scenarios may cover initialization, data transmission, and data reception, etc.

Given a verification scenario, not all part of the RTL model is exercised. As shown in Fig. 1, when verifying the ADD instruction, logic related to other instructions can be safely abstracted. However, conventional techniques including the Cone of Influence may miss such opportunities because they rely on structural rather than functional analysis. Consequently, we propose using LLMs to identify the logic that is truly relevant to each verification scenario.

3 Our Method

3.1 Workflow Overview

Fig. 2 illustrates the abstraction process in NeuroAbs. Rather than allowing the LLM to directly rewrite RTL in a free-form manner, NeuroAbs follows a neuro-symbolic flow in which the neural component proposes scenario-aware abstraction candidates and the symbolic component constrains, validates, and refines them. NeuroAbs proceeds in three stages. First, the RTL Signal Analyzer takes as input the design specification, the verification scenario, and signal definitions, and identifies signals that are likely relevant to the target property. Second, starting from the selected signals, NeuroAbs locates the corresponding RTL statements, maps them to AST statement nodes, and asks the Abstractor to generate candidate abstractions only at the selected rewrite points. These candidate rewrites are then checked by SMT, and only those that satisfy the required over-approximation are applied to update the abstract design. Third, the resulting abstract design is verified against the

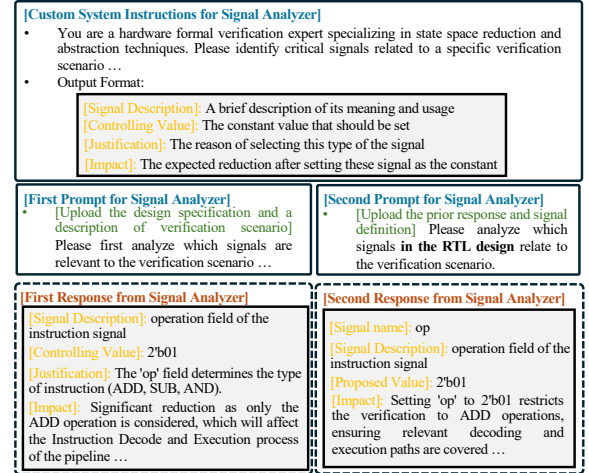


Figure 3: The custom instruction, COT prompt and response of the Signal Analyzer.

target property. If the abstraction is still too coarse and introduces spurious counterexamples, NeuroAbs uses a customized CEGAR loop with counterexample reduction to refine the abstract model.

3.2 Candidate Signal Identification for Abstraction

As RTL can be very lengthy, it is paramount to first narrow down where abstraction may be applied. The goal of this stage is to identify candidate signals for abstraction, rather than determine the final rewrite itself. A naive approach would be to provide the entire RTL code to an LLM for analysis. However, given the limited context length of LLMs, directly feeding thousands of lines of RTL code for abstraction is infeasible.

To address this problem, we propose a hybrid approach that integrates an LLM with RTL static analysis to identify abstraction candidates. The LLM analyzes the design specification and verification scenario description, along with formal properties and environment setup when available, to extract relevant information in natural language that traditional rule-based methods cannot easily capture. It then suggests signals related to the scenario, referred

to as *scenario-related signals*. Starting from these signals, RTL static analysis is performed to further track the corresponding RTL logic where abstraction may later be applied. At this stage, the output is a localized set of candidate signals for abstraction, rather than a concrete rewrite.

Inspired by chain-of-thought (COT) [26], we build an LLM-based pipeline, called the Signal Analyzer, to generate the *scenario-related signals* in two steps. First, it interprets signal behaviors based on the design specifications and the scenario descriptions. Then, it outputs specific signal names according to the prior analysis and the signal definitions in the RTL model. Fig. 3 shows a brief example of the Signal Analyzer’s prompts and responses. This LLM is guided by customized system instructions containing background information of RTL verification, the general setting of NeuroAbs, and the subsequent verification steps. We also provide a unified template for the initial response, which directs the LLM to describe each signal’s meaning and controllable value within the verification scenario. The template also requires the LLM to justify its signal selection and explain the expected effect of simplifying the related logic, enabling engineers to better understand the LLM’s choices.

After generating the initial response, the next step of the Signal Analyzer is to map the scenario-related signals in the documents to their actual names in RTL. We provide the LLM with the signal definitions in the RTL module together with its first response. As shown in Fig. 3, the analyzer aligns the signal declaration with the generated descriptions, and outputs the response accordingly.

Given the initial set of scenario-related signals, we apply RTL static analysis to identify the corresponding code regions relevant to these signals, such as statements connected through control or data dependency. In some verification scenarios, certain control signals are assigned or assumed to hold fixed values. The analysis can also identify opportunities of simplification due to fixed values through techniques such as constant propagation [25]. Note that these fixed values are used only in this static-analysis stage to guide localization. Once the abstraction targets are identified, they are not carried forward as additional assumptions in the generated abstract model or the downstream verification procedure.

Fig. 4 presents a simplified example based on the same microprocessor design shown in Fig. 1. When verifying the ADD instruction, the Signal Analyzer identifies that the opcode field of the instruction word (*inst*) is a scenario-related signal with a controlled value of $2'b01$, corresponding to the `OP_ADD` operation. While the remaining bits indicating source registers and the destination register are not controlled. By applying constant propagation, this constant value is propagated to related signals, allowing certain assignments — such as those to *id_wen* and *alu_result* — to be simplified, as illustrated in the example. This simplification suggests that the corresponding statements of these signals are suitable candidates for abstraction, consistent with the earlier discussion in Section 1. This step narrows the abstraction scope to a small set of candidate signals, which are then used to localize statement-level AST nodes in the next subsection.

3.3 Neuro-Symbolic LLM-based Abstraction

The previous stage has already identified the signals that are relevant to the target property. We next convert RTL into an AST-based

```

1 module pipeline(input clk, input rst, input
2   [7:0] inst, ...);
3   wire [1:0] op, rs1, rs2, rd; reg [7:0] val [0:3];
4   wire id_wen; wire [7:0] alu_result;
5   assume property (inst == {2'b01, 2'bx, 2'bx, 2'bx});
6   // Before Constant Propagation
7   assign op = inst[7:6]; assign rs1 = inst[5:4];
8   assign rs2 = inst[3:2]; assign rd = inst[1:0];
9   // After Constant Propagation
10  assign op = 2'b01; assign rs1 = 2'bx;
11  assign rs2 = 2'bx; assign rd = 2'bx;
12  // Before Constant Propagation
13  assign id_wen = op == OP_ADD || op ==
14    OP_SUB || op == OP_AND;
15  // After Constant Propagation
16  assign id_wen = 1'b1;
17  // Before Constant Propagation
18  assign alu_result = op == OP_ADD ?
19    val[rs1] + val[rs2] : op == OP_SUB ?
20    val[rs1] - val[rs2] : op == OP_AND ?
21    val[rs1] & val[rs2] : 8'bxxxxxxx;
22  // After Constant Propagation
23  assign alu_result = val[rs1] + val[rs2];
24  assert property (...); // Whether the instruction
   ADD can be correctly decoded and executed;

```

Figure 4: The example of constant propagation. After we assign a constant value to *inst*, the signal *op* is set to `OP_ADD` ($2'b01$). Following the constant propagation, *id_wen* and *alu_result* will evaluate to $1'b1$ and $val[rs1] + val[rs2]$.

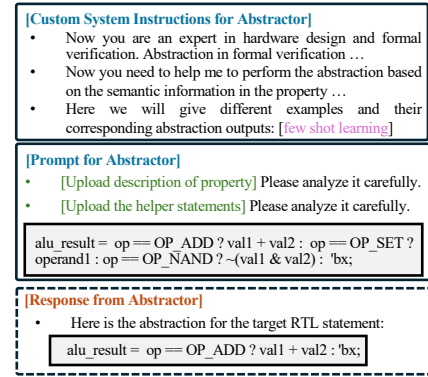


Figure 5: The custom instruction, prompt and response of the Abstractor.

symbolic representation and traverse the AST to localize the statement(s) corresponding to these signals. This AST representation provides a precise structural boundary for each candidate statement compared to unconstrained text-level localization in RTL, recovers the enclosing RTL context, and links each rewrite to the subsequent symbolic checking and refinement steps. The Abstractor then generates abstractions only at these selected nodes, and each resulting rewrite is checked to ensure the required over-approximation relation.

AST-based localization. Let A denote the set of AST nodes in the RTL design, and let:

$$\Phi_{loc} : A \rightarrow \{true, false\}$$

be the matching condition induced by the localization result from the previous stage. For a node $a \in A$, $\Phi_{loc}(a) = true$ means that a is a statement-level AST node localized from the selected signals. These nodes serve as the symbolic interface that constrains the neural Abstractor to a finite set of semantically meaningful rewrite targets. We further trace upward to the enclosing constructs, such as

“always” blocks, “initial” blocks, and “case” statements, and provide them to the Abstractor as auxiliary structural context.

Transformation. After the target statement is fixed, the second LLM, called Abstractor, performs the transformation. We ask the Abstractor to generate the abstraction on the selected statement-level AST node together with its local structural context. Given this statement and its enclosing structural context, the Abstractor rewrites the corresponding RTL text only within the AST-localized statement region, rather than editing an unconstrained text span. The Abstractor then decides which internal part of the statement should be abstracted according to the property. Formally, let:

$$\tau : \{a \in A \mid \Phi_{loc}(a) = true\} \rightarrow \hat{A}$$

denote the abstraction transformation, where τ maps each localized statement AST node a to a rewritten AST $\hat{a} \in \hat{A}$. As illustrated in Fig. 5, the Abstractor receives the relevant property specification, the entire target statement, and its enclosing structural context, and rewrites only the part of that statement judged irrelevant to the target property. For example, in the statement computing `alu_result`, the addition-related computation is preserved, while the unrelated part can be abstracted with X-values or simplified expressions. In this way, the AST determines *which* statement may be rewritten, while the LLM determines *how* to rewrite the statement.

Besides the target RTL statement and its context, the Abstractor employs few-shot learning [23] by providing a small set of input-output examples (original vs. abstract statements) to increase the probability of generating a correct abstraction. The soundness of each generated abstraction is then checked by an SMT-based rule, as described next.

3.4 SMT-based Soundness Constraint

Because the LLM-generated abstraction may fail to over-approximate the original RTL design, we impose an additional symbolic constraint using Satisfiability Modulo Theories (SMT) [1]. For each selected RTL statement, the generated abstraction is accepted only if it satisfies the following SMT checking rule:

$$\forall V, S(V) \Rightarrow \hat{S}(V) \quad (1)$$

where $S(V)$ and $\hat{S}(V)$ denote the SMT expressions induced by the original and abstracted versions of the selected RTL statement, respectively. This rule requires every behavior allowed by the original statement to also be allowed by the abstracted statement, i.e., the abstraction must be an over-approximation. In many cases, the Abstractor introduces unknown values (X-values) in the abstract statements. Existing SMT solvers do not natively support X-values. To address this issue, we translate each X-value into a fresh variable bound by an existential quantifier, and refine the checking rule as follows:

$$\forall V, \exists Y, [S(V, Y) \Rightarrow \hat{S}(V, Y)] \quad (2)$$

Here, Y denotes the additional variables introduced for the X-values. This formulation lets the SMT solver interpret the X-values as non-deterministic choices and check whether the rewritten statement still over-approximates the original statement. Although quantified bit-vector reasoning is generally harder than quantifier-free solving, the formulas in Equation 2 remain moderate in size in our setting.

Therefore, we do not observe performance issues in soundness checking in our experiments. Since each accepted rewrite over-approximates its original local RTL statement and all unrevised statements remain unchanged, the composed abstract RTL model also remains an over-approximation of the original design.

3.5 Customized CEGAR Loop

Once the abstract model is built through the prior steps, the model could be utilized to verify the target property $P(V)$. However, as discussed in Section 2.1, abstraction may incur spurious counterexamples if it is too coarse for the property to verify. Consequently, the model needs to be refined to eliminate these spurious counterexamples. Prior research has proposed the counterexample-guided abstraction refinement (CEGAR) paradigm to iteratively block these spurious counterexamples [9]. However, a coarse abstraction often leads to numerous counterexamples, and it may take many iterations to properly refine the model. It is beneficial if the counterexample could be generalized to better guide the refinement. Therefore, we propose a customized CEGAR implementation integrated with counterexample reduction and generalization techniques [29] to better pinpoint which rewritten statement(s) actually trigger the violation of the property. These relevant abstract statements are then reverted to their original form. By focusing on refining abstract statement(s) that trigger a set of counterexample, this approach helps to reduce the number of iterations in the CEGAR loop. We name this implementation as the R-CEGAR method to emphasize the use of counterexample reduction techniques.

Fig. 6 illustrates an example of counterexample reduction in our setting. This example is derived from the abstract statement in Fig. 1 (lines 19–20), which computes the value of the signal `alu_result`. The statement involves a 2:1 multiplexer controlled by a comparator that outputs 1 during the execution of the addition operation. As mentioned earlier, we conjecture that operations related to subtraction and bitwise AND do not contribute to the property that we want to check. Therefore, we abstract these operations by replacing them with X-values. For each abstraction, we automatically generate an auxiliary input (labeled as `aux_input` in the figure) for the X-value. As shown in the counterexample trace, the multiplexer control bit is 0, meaning the result of `alu_result` depends solely on the signal `aux_input`, without considering the addition of `val[rs1]` and `val[rs2]`. As a result, after applying counterexample reduction, it becomes clear that only `aux_input` is relevant to the property violation, indicating that the introduced X-value is not a proper abstraction for the properties. This indicates that the abstraction has introduced a spurious counterexample, requiring us to revise this abstraction and revert to the original statement.

The overall R-CEGAR process is illustrated in Algorithm 1. The algorithm takes as input the initial abstract state transition system $\hat{M}$ and the target property P . In each iteration, the algorithm performs property checking (line 2). If a violation is detected, it extracts a counterexample and applies the reduction technique to generalize the trace (line 6). It then checks if any abstract statements contribute to the property violation (line 7). If such statements are detected, we first locate them through the AST-based symbolic representation (line 11), mapping each invalid abstract statement back to its corresponding RTL region and rewrite point, and then revise them

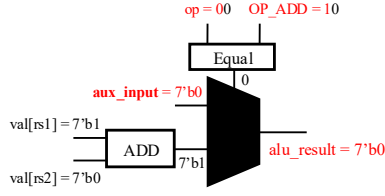


Figure 6: An example of counterexample reduction for our abstract model. When the target property is violated, assignments can be extracted. The reduced counterexample is highlighted in red.

accordingly (line 12); otherwise, the property is indeed violated, implying that the original model also fails under this property (line 9).

4 Experiment

In this section, we aim to answer the following questions through a comparative evaluation of NeuroAbs: **(RQ1)** What is the performance of NeuroAbs in generating the abstract statements? **(RQ2)** Can the abstraction accelerates model checking algorithms in the construction of formal proofs? **(RQ3)** When model checking algorithms can not conclude with a formal proof, can our abstraction method help to explore more state transitions for bug-finding?

4.1 Experiment Setup

The experiments are conducted on a machine with a 2.9 GHz Intel Xeon Platinum 8375C CPU and 256 GB RAM. We adapt the open-source RTL synthesis tool Yosys to perform RTL static analysis and use Pyverilog to transform the RTL code to AST. For the LLM, we utilize the gpt-4o-mini-2024-07-18’s API to perform the abstraction since it balances the cost and the quality of the generated result. For few-shot learning, we provide 8 input-output examples for NeuroAbs to learn how to perform abstraction. We translate the rewritten RTL statements into symbolic formulas and use the SMT solver Z3 to check whether each generated abstraction satisfies the required over-approximation constraint.

We integrate our abstraction method with two model checkers: RIC3 [22] and AVR [14]. AVR was the champion of the 2020’s hardware model checking competition (HWMCC) [20]. It incorporates various word-level model checking algorithms and supports parallel execution. We utilize its development branch for our experiments. RIC3 won the championship at the 2024’s HWMCC [4]. It includes

Algorithm 1: The R-CEGAR process

```

Input:  $\hat{M}$ : the abstract model generated by our method,  $P$ : the target property to prove;
Output:  $res \in \{SAFE, UNSAFE\}$ ;
1 while  $True$  do
2    $cex \leftarrow \text{PropertyChecking}(\hat{M}, P)$ ;
3   if  $cex = \emptyset$  then
4     // The property is proved.
5     return SAFE
6   else
7      $cex\_generalized \leftarrow \text{Generalize}(cex)$ 
8      $invalid\_statements \leftarrow \text{Check}(cex\_generalized)$ 
9     if  $invalid\_statements = \emptyset$  then
10      return UNSAFE
11    else
12       $regions \leftarrow \text{Locate}(invalid\_statements)$ 
13       $\hat{M} \leftarrow \text{Refine}(\hat{M}, invalid\_statements, regions)$ 
14    end
15 end

```

Table 1: The statistics of benchmark designs in the experiment.

Source Benchmarks	Verification Task	Number of Scenarios	# RTL Lines	# Input Vars	# State Vars
PicoRV32	Generic Inst.-specific	8	3079	82	209
Piccolo	Generic Inst.-specific	22	14943	219	381
Flute	Generic Inst.-specific	22	17228	279	407
I2C	Reg-interface	3	1257	157	47

Table 2: Ablation study of NeuroAbs

	OA rate	Degradation	# avg. strict OA	Degradation
NeuroAbs	95.27%	-	236.17	-
w.o. few-shot	83.51%	12.34%	136.00	42.41%
w.o. few-shot & custom inst.	96.46%	-1.25%	3.83	98.38%

* A rewritten statement is an over-approximation (OA) as long as it complies with Eq. (2). This does not rule out equivalent statements.

† This column reports the averaged number of strict over-approximated statements, which must comply with both Eq. (2) and Eq. (3).

several bit-level model checking algorithms running in parallel. We used the version prepared for the 2024’s competition in our experiments. Note that both AVR and RIC3 integrate a variety of advanced abstraction techniques and have demonstrated highly competitive performance in HWMCC, making them substantially stronger baselines than prior abstraction approaches such as IC3IA. Our goal is to demonstrate that NeuroAbs can further improve the efficiency of these model checkers, even when they are already equipped with traditional abstraction mechanisms.

4.2 Benchmarks

For our experiments, we need benchmark designs that come with: 1) the RTL implementation, 2) a design specification, and 3) a set of formal properties to be verified. However, many existing open-source RTL designs lack a good design specification in the first place and few are equipped with formal properties. Therefore, we turn to the verification tasks of open-source RISC-V processors, as RISC-V processor designs are based on well-defined standards, and there are existing works providing formal properties to verify. The verification tasks we use are over three processor cores, PicoRV32 [30], Piccolo [6] and Flute [5]. PicoRV32 is verified using the RISC-V formal framework [24], while Piccolo [6] and Flute [5] are verified using the instruction-level abstraction (ILA) refinement checking [17]. To demonstrate applicability beyond processors, we also include an I2C peripheral, verifying properties such as read-after-write consistency in its register interface. Table 1 summarizes the statistics of our benchmarks, which shows that the benchmarks used in experiments are non-trivial in terms of RTL code size, with a total of 55 verification scenarios identified across three designs. Among these verification scenarios, some check the correctness of a single instruction or an instruction class (instruction-specific), while others check generic properties of the RTL, the sanity of the specified properties, or register-interface consistency.

4.3 Effectiveness of our LLM in Performing Abstraction

To address **RQ1**, we conduct an ablation study to evaluate the contribution of individual components in the NeuroAbs framework

Table 3: Average unrolling depths explored by BMC w./w.o. NeuroAbs.

Design*	Runtime (s)	With NeuroAbs	Baseline†	Time to Reach Baseline Max Depth (s)	Improvement (%)
Piccolo	23600	125.36	109.36	15482.67	34.40%
Flute		225.09	129.00	5668.58	75.98%

† Baseline refers to the results without NeuroAbs.

* Since all PicoRV32-related and I2C-related cases have been formally proven in the previous experiment, we do not include them in the BMC experiments.

for abstraction construction. Specifically, we compare the full NeuroAbs against degraded variants in which certain design elements are removed step by step. First, we disable few-shot learning, thereby removing example-based guidance that helps the LLM produce correct abstract statements. Then, we remove the custom instruction prompt, leaving the model without sufficient contextual understanding of the abstraction task. This configuration corresponds to the default behavior of gpt-4o-mini-2024-07-18.

We evaluate performance using two metrics. The first is the over-approximation rate (OA rate), which measures the percentage of generated statements that correctly produce an over-approximate version, as validated by the condition in Eq. (2). The second metric is the average number of strict over-approximations. In addition to satisfying the over-approximation condition, this metric requires that the abstracted statement be semantically different from the original, such that it is not trivially equivalent to the source statement. Formally, the abstract statement must satisfy both the condition in Eq. (2) and the following criterion:

$$\forall V, \exists Y, [S(V, Y) \neq \hat{S}(V, Y)] \quad (3)$$

Table 2 presents the overall results of the ablation study. From this table, two key observations can be made. First, removing the few-shot learning mechanism significantly reduces the model’s ability to generate correct over-approximated statements, as it lacks concrete guidance on how to construct valid abstractions. This leads to a decreased OA rate (83.51%). Second, when custom instructions are also removed, the OA rate appears to increase slightly to 96.46%. However, this improvement is misleading. Without the custom instructions, NeuroAbs loses awareness of the objective of making an abstraction and instead tends to greedily reproduce the original statements. As a result, the number of strict over-approximation drops drastically, with an average of only 3.83 statements—representing a 98.38% decrease compared to the fully configured NeuroAbs. Overall, these results emphasize the importance of both few-shot learning and custom instruction prompts: together, they enable NeuroAbs to produce meaningful and diverse abstractions.

4.4 Effectiveness of Abstraction in Concluding Formal Proofs

To address RQ2, we evaluate the effectiveness of our abstraction technique in facilitating formal proofs. Fig. 7 compares the verification runtime with and without NeuroAbs across different model checkers, with a 3600-second timeout for each case. When integrated with AVR, NeuroAbs significantly reduces proof time, achieving an overall runtime improvement of 44.89%. Although AVR already includes advanced abstraction mechanisms such as syntax-guided abstraction and uninterpreted function abstraction,

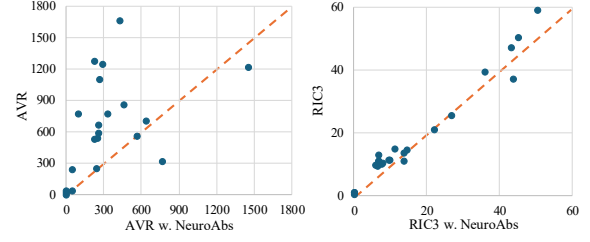


Figure 7: The runtime (measured in seconds) comparison of reaching a formal proof with or without NeuroAbs. The mean values are: AVR = 607.03, AVR w. NeuroAbs = 334.56; RIC3 = 18.71, RIC3 w. NeuroAbs = 16.94.

NeuroAbs further complements them and enhances performance. For RIC3, integration also improves runtime, though the gains are marginal as RIC3 is already running fast for these cases.

4.5 Effectiveness of Abstraction in BMC

For cases where a formal proof cannot be concluded in reasonable time, we analyze the effectiveness of our abstraction method in accelerating the BMC algorithm to address RQ3. Although these cases do not yield a formal proof, faster BMC runs allow deeper exploration within the same runtime, improving the chance of finding bugs or gaining confidence in correctness. Since AVR is not as competitive as RIC3, as shown by the prior result, this experiment solely utilizes the BMC implementation in RIC3 for comparison. Table 3 presents the average achieved depths with and without NeuroAbs. We allow BMC to run for up to 23600 seconds per case to capture meaningful differences in the depths reached by each configuration. In this table, the column “Time to Reach Baseline Max Depth” indicates the time required for BMC with NeuroAbs to reach the maximum cycles achieved by the baseline.

As shown in Table 3, integrating NeuroAbs into the BMC algorithm substantially enhances its efficiency, enabling deeper exploration within the same runtime. For example, in the “Piccolo” case, the average depth increases from 109.36 to 125.36. It also greatly reduces the time required to reach the baseline’s maximum depth; in the “Flute” case, the runtime decreases by 75.98%, from 23,600 seconds to 5,668.58 seconds. Detailed results for representative cases are also provided in Table 4. Overall, NeuroAbs consistently improves BMC performance across the given tasks.

Table 4 also reports the total runtime, the number of refinement iterations, the cumulative LLM runtime, and the average time required for the LLM to abstract each statement. The results show that the initial abstract model can be refined efficiently, requiring only a small amount of time and few iterations. On average, the refinement process takes 7.26 seconds and 1.65 iterations. This efficiency stems from the high accuracy of NeuroAbs, which produces high-quality abstractions that greatly reduce the refinement effort.

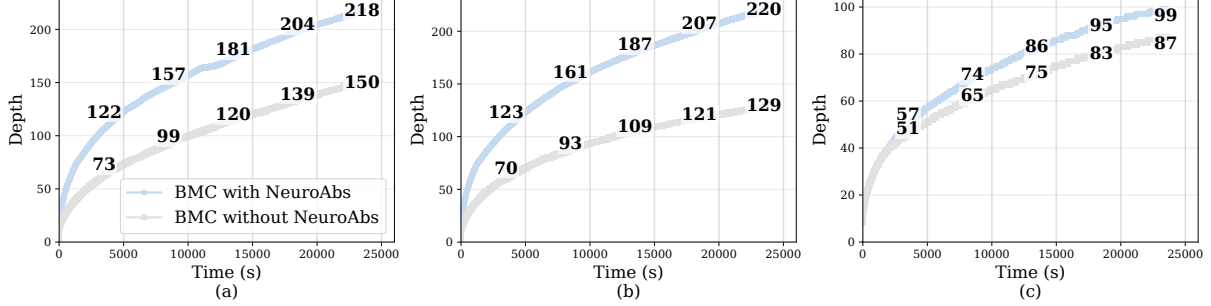
Furthermore, we plot the wall-clock time against the number of reached bounds for three representative cases, as shown in Fig. 8. From this figure, we observe that as the time increases, the gap between the depths reached by BMC with and without NeuroAbs continues to widen. For example, in the middle graph, at 10,000 seconds, the depth difference is 68 (161 vs. 93), but by 20,000 seconds, this gap increases to 86 (207 vs. 121). This trend demonstrates the effectiveness of the abstraction in simplifying the circuit structure,

Table 4: Experiment results on representative cases for BMC with or without NeuroAbs.

Benchmark	Verification Task	# Variable Index (w./w.o. Abstraction)	Total LLM time (s) *	Avg. LLM time / stmt (s)	CEGAR time (s)	# CEGAR Iter	# Bound with NeuroAbs	# Bound Baseline	Time to Reach Baseline Max Bound (s)	Improvement (s)
Piccolo	OR-ILA	118188/129316	361.79	4.41	2.88	2	99	87	15378.68	34.84%
	ANDI-ILA	116128/119628	275.80	3.40	4.05	3	123	106	14126.45	40.14%
	JAL-ILA	111567/116969	305.72	3.40	0.62	1	190	163	13982.43	40.75%
	LUI-ILA	111546/115882	274.46	3.19	5.79	3	200	180	15396.60	34.76%
Flute	SUB-ILA	299217/1273706	632.07	2.13	6.83	1	221	138	6978.98	70.43%
	XORI-ILA	295565/1272600	611.42	2.09	13.77	1	224	133	5351.54	77.32%
	BLTU-sanity	306462/1278074	663.89	2.31	42.00	2	216	123	5252.36	77.74%
	BGE-sanity	300650/1275355	604.23	2.04	14.26	2	220	129	5552.50	76.47%

[†] Due to page limits, we are unable to include all detailed results in this table. Additional results are available at: [\[Link\]](#)

* The reported time in this column includes the time spent on LLM inference, SMT solving, and AST construction and traversal.

**Figure 8: Comparison of Depth vs. Time relationship between BMC with and without NeuroAbs across three representative examples.**

which reduces the complexity of the unrolled expressions in BMC. As a result, the solving process becomes increasingly efficient over time, particularly at larger unrolling depths.

Regarding the inference time for each statement, the LLM demonstrates efficient abstraction performance, requiring on average 3.70 seconds to rewrite a statement. The total LLM runtime, however, is relatively longer because abstractions are generated sequentially rather than in parallel. Nevertheless, even when accounting for both LLM runtime and CEGAR refinement time, our method still outperforms the baseline. Further speedups could be achieved by parallelizing the rewriting process to avoid linear growth in total runtime.

4.6 Discussion

To understand why NeuroAbs improves the efficiency of model checkers, we compare hardware sizes with and without our abstraction technique for the same representative cases shown in Table 4. Specifically, we translate the RTL design into an And-Inverter-Gate (AIG) circuit [2] using Yosys [28], and record the maximum variable index, as summarized in Table 4. In the AIG format, the maximum variable index equals the sum of inputs, latches, and AND gates. The results show that the maximum variable index are greatly reduced. This reduction comes from the way our abstraction simplifies large parts of the original logic, thereby lowering circuit complexity and accelerating the model checking algorithm.

4.7 Supported Types of RTL Constructs

We further summarize the types of RTL constructs that NeuroAbs can abstract based on the observations. As shown in Table 5, NeuroAbs supports several common abstraction types in RTL designs. It successfully abstracts statements with unary and binary operators from non-blocking assignments and entire always blocks containing multiple blocking statements. It also flexibly abstracts branch

Table 5: Common RTL Abstraction Types.

Type	Before NeuroAbs	After NeuroAbs
Unary	<code>c = ~a;</code>	<code>c = 'bx;</code>
Binary	<code>out = a & b;</code>	<code>out = 'bx;</code>
Ternary	<code>out = (PIPELINED) ? a + b : a - b;</code>	<code>out = (PIPELINED) ? a + b : 'bx;</code>
Case	<code>case (inst) begin</code> <code>1'b1: result = a b;</code> <code>1'b0: result = a*b; end</code>	<code>case (inst) begin</code> <code>1'b1: result = a b;</code> <code>1'b0: result = 'bx; end</code>
For	<code>for (i = 0; i < 32; i = i+1)</code> <code>regs = 0;</code>	<code>for (i = 0; i < 32; i = i+1)</code> <code>regs = 'bx;</code>
Always	<code>always @(posedge clk)</code> <code>begin compare <= {beq, bne};</code> <code>ari <= {add, sub}; end</code>	<code>always @(posedge clk)</code> <code>begin compare <= {beq, bne};</code> <code>ari <= 'bx; end</code>
Initial	<code>initial begin</code> <code>receive <= 'b0; send <= 'b0; end</code>	<code>initial begin</code> <code>receive <= 'bx; send <= 'bx; end</code>

structures, such as ternary operators and case statements, according to the given property. These results show that NeuroAbs effectively handles diverse abstraction scenarios.

5 Conclusion

In this paper, we introduce NeuroAbs, a neuro-symbolic abstraction framework that leverages LLMs to achieve flexible and automated abstraction for RTL models. The framework starts with LLM-assisted RTL analysis to identify suitable signals and statements for abstraction, followed by AST-guided LLM-based abstraction to generate an initial abstract model. Finally, SMT checking and the CEGAR process are employed to maintain soundness and iteratively refine the model when necessary. Experimental results demonstrate that NeuroAbs can effectively improve the efficiency of formal property verification on various RTL designs.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (grant no. 62304194).

References

- [1] Clark Barrett and Cesare Tinelli. 2018. Satisfiability modulo theories. *Handbook of model checking* (2018), 305–343.
- [2] Armin Biere. 2007. The AIGER and-inverter graph (AIG) format version 20071012. (2007).
- [3] Armin Biere, Alessandro Cimatti, Edmund Clarke, and Yunshan Zhu. 1999. Symbolic model checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems: 5th International Conference, TACAS'99 Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS'99 Amsterdam, The Netherlands, March 22–28, 1999 Proceedings* 5. Springer, 193–207.
- [4] Armin Biere, Nils Froyleyks, and Mathias Preiner. 2024. Hardware Model Checking Competition 2024. In *# PLACEHOLDER_PARENT_METADATA_VALUE#*. TU Wien Academic Press, 7–7.
- [5] Inc Bluespec. 2018. Flute, a free and open-source RISC-V CPU. <https://github.com/bluespec/Flute>.
- [6] Inc Bluespec. 2018. Open-source RISC-V CPUs from Bluespec, Inc. <https://github.com/bluespec/Piccolo>.
- [7] Randal E Bryant, Shuvendu K Lahiri, and Sanjit A Seshia. 2002. Modeling and verifying systems using a logic of counter arithmetic with lambda expressions and uninterpreted functions. In *International Conference on Computer Aided Verification*. Springer, 78–92.
- [8] Alessandro Cimatti, Alberto Griggio, Sergio Mover, and Stefano Tonetta. 2014. IC3 modulo theories via implicit predicate abstraction. In *Tools and Algorithms for the Construction and Analysis of Systems: 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5–13, 2014. Proceedings* 20. Springer, 46–61.
- [9] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-guided abstraction refinement. In *Computer Aided Verification: 12th International Conference, CAV 2000, Chicago, IL, USA, July 15–19, 2000. Proceedings* 12. Springer, 154–169.
- [10] Niklas Eén, Alan Mishchenko, and Robert Brayton. 2011. Efficient implementation of property directed reachability. In *2011 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE, 125–134.
- [11] Hongyu Fan and Fei He. 2024. Leveraging Datapath Propagation in IC3 for Hardware Model Checking. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).
- [12] Kuan Fan, Ming-Jen Yang, and Chung-Yang Huang. 2016. Automatic abstraction refinement of TR for PDR. In *2016 21st Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 121–126.
- [13] Wenji Fang, Guangyu Hu, and Hongce Zhang. 2023. r-map: Relating Implementation and Specification in Hardware Refinement Checking. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (2023), 5113–5126.
- [14] Aman Goel and Karem Sakallah. 2020. AVR: abstractly verifying reachability. In *Tools and Algorithms for the Construction and Analysis of Systems: 26th International Conference, TACAS 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25–30, 2020. Proceedings, Part I* 26. Springer, 413–422.
- [15] Yen-Sheng Ho, Alan Mishchenko, and Robert Brayton. 2017. Property directed reachability with word-level abstraction. In *2017 Formal Methods in Computer Aided Design (FMCAD)*. IEEE, 132–139.
- [16] Y-W Hsieh and Steven P Levitan. 1998. Model abstraction for formal verification. In *Proceedings Design, Automation and Test in Europe*. IEEE, 140–147.
- [17] Bo-Yuan Huang, Hongce Zhang, Pramod Subramanyan, Yakir Vitzel, Aarti Gupta, and Sharad Malik. 2018. Instruction-level abstraction (ila) a uniform specification for system-on-chip (soc) verification. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 24, 1 (2018), 1–24.
- [18] Chung-Wah N Ip, Lawrence Loh, Vigyan Singhal, and Howard Wong-Toi. 2008. Managing formal verification complexity of designs with counters. US Patent 7,418,678.
- [19] Alan Mishchenko, Niklas Een, Robert Brayton, Jason Baumgartner, Hari Mony, and Pradeep Nalla. 2013. GLA: Gate-level abstraction revisited. In *2013 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1399–1404.
- [20] Mathias Preiner, Armin Biere, and Nils Froyleyks. 2020. Hardware model checking competition 2020. (2020).
- [21] Mary Sheeran, Satnam Singh, and Gunnar Stålmarck. 2000. Checking safety properties using induction and a SAT-solver. In *International conference on formal methods in computer-aided design*. Springer, 127–144.
- [22] Yuheng Su, Qiusong Yang, Yiwei Ci, Tianjun Bu, and Ziyu Huang. 2025. The rIC3 hardware model checker. In *International Conference on Computer Aided Verification*. Springer, 185–199.
- [23] Yaqing Wang, Quanming Yao, James T Kwok, and Lionel M Ni. 2020. Generalizing from a few examples: A survey on few-shot learning. *ACM computing surveys (csur)* 53, 3 (2020), 1–34.
- [24] Andrew Waterman, Yunsup Lee, David A Patterson, and Krste Asanovic. 2014. The RISC-V instruction set manual, volume I: User-level ISA, version 2.0. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-54* (2014), 4.
- [25] Mark N Wegman and F Kenneth Zadeck. 1991. Constant propagation with conditional branches. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 13, 2 (1991), 181–210.
- [26] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [27] Claire Wolf. 2024. <https://github.com/YosysHQ/riscv-formal>
- [28] Clifford Wolf et al. 2016. Yosys open synthesis suite.
- [29] Zhiyuan Yan and Hongce Zhang. 2025. Word-Level Counterexample Reduction Methods for Hardware Verification. In *2025 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
- [30] YosysHQ. 2015. PicoRV32 - A Size-Optimized RISC-V CPU. <https://github.com/YosysHQ/picorv32>.